rijksuniversiteit
groningen

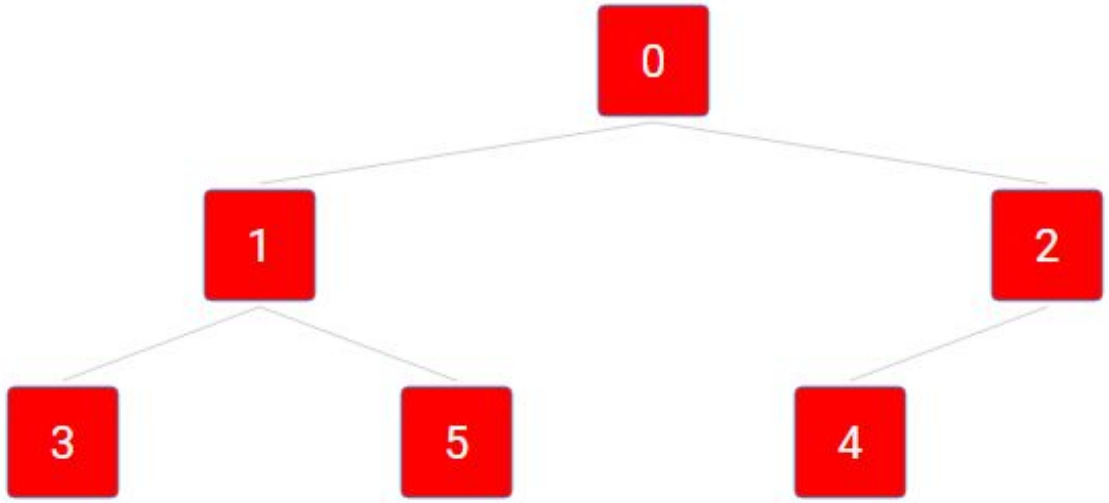
Formal Proofs and Algorithmic Analysis of Braun Trees in Coq

Author: Alexandru Filiuta

Thesis Supervisor: Dan Frumin

Table of Contents

What are Braun Trees



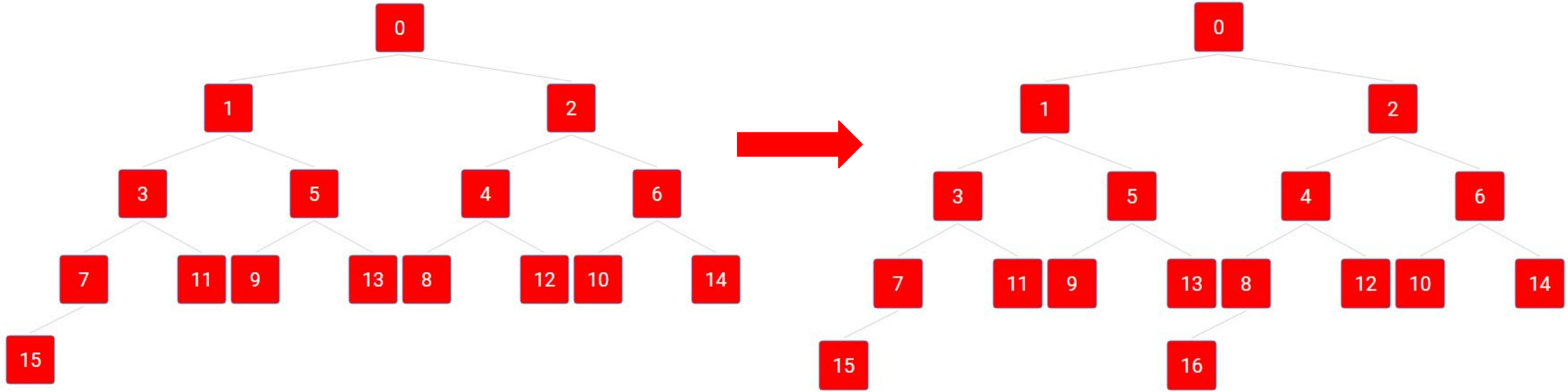
"Braun trees for extensible arrays were first investigated by Braun and Rem [1983] and, in a more functional setting, by Hoogerwoord [1992]."

https://isabelle.in.tum.de/~nipkow/pubs/cpp20-official.pdf

Properties

- Binary lookup trees
- Balanced structure
- Recursive nature
- Uniform shape
- Efficient indexing

Inserting a new element in a Braun Tree of size 16, with each node labelled by its index



Motivation & Usefulness

1. Why are Braun Trees useful?
   1.1. Efficient implementation of DS
   1.2. Logarithmic depth **->** Logarithmic performance
   1.3. Simple recursive operations
2. Practical Applications
   2.1. Priority Queues
   2.2. Flexible Arrays
3. Motivation for Formal Verification
   3.1. Limited scientific coverage
   3.2. Guarantees reliability and accuracy

Braun Tree datatype

```
Inductive BraunTree (V: Type) : Type :=
  | Empty
  | Braun (l: BraunTree V) (v: V) (r: BraunTree V).

  Arguments Empty {V}.
  Arguments Braun {V} _ _ _.
```

Basic Operations

- General Operations: Size, Replicate

```
Fixpoint sizeOrg {V: Type} (t: BraunTree V) : nat :=
  match t with
  | Empty => 0
  | Braun l _ r => 1 + sizeOrg l + sizeOrg r
  end.
```

```
Fixpoint replicate {V: Type} (x: V) (n: nat) : BraunTree V :=
  match n with
  | 0 => Empty
  | S n' => insert x (replicate x n')
  end.
```

- For **Priority Queues** and **Heaps**: Insert, Remove

```
Fixpoint insert {V: Type} (v: V) (t: BraunTree V) : BraunTree V :=
  match t with
  | Empty => Braun Empty v Empty
  | Braun l v' r => if Nat.eqb (sizeOrg l) (sizeOrg r) then
    Braun (insert v l) v' r else Braun l v' (insert v r)
  end.
```

```
Fixpoint removeRoot {V: Type} (t: BraunTree V) : option (V *
BraunTree V) :=
  match t with
  | Empty => None
  | Braun Empty v Empty => Some (v, Empty)
  | Braun l v r =>
    match removeRoot l with
    | Some (lv, newL) => Some (v, Braun r lv newL)
    | None => None (* UNREACHABLE CASE *)
    end
  end.
```

Basic Operations (cont.)

› For **Dynamic Arrays**: Lookup, Update, Tree to List

```
Fixpoint lookup {V: Type} (t: BraunTree V) (n: nat) :
option V :=
  match t with
  | Empty => None
  | Braun l v r =>
      match n with
      | 0 => Some v
      | S m => if Nat.even m
               then lookup l (Nat.div2 m)
               else lookup r (Nat.div2 m)
      end
  end.

(* Helper lemma *)
Fixpoint merge_lists {V: Type} (l1 l2: list V) : list V :=
  match l1, l2 with
  | [], _ => l2
  | _, [] => l1
  | x::xs, y::ys => x :: y :: (merge_lists xs ys)
  end.
```

```
Fixpoint update {V: Type} (n: nat) (v: V) (t: BraunTree V) :
BraunTree V :=
  match t with
  | Empty => Empty
  | Braun l v' r =>
      match n with
      | 0 => Braun l v r
      | S m =>
          if Nat.odd n
          then Braun (update (Nat.div2 m) v l) v' r
          else Braun l v' (update (Nat.div2 (n - 1)) v r)
      end
  end.

Fixpoint tree_to_list {V: Type} (t: BraunTree V) : list V :=
  match t with
  | Empty => []
  | Braun l v r => v :: (merge_lists (tree_to_list l)
(tree_to_list r))
  end.
```

Formal Proving

	Formal Proofs Proofs executed within a formal logical system	Normal Proofs
Formality level	› Explicit detailing of every logical step and adherence to strict logical rules	› Normal proofs may use natural language and assume that the reader accepts common logical inferences, often leaving some logical leaps
Tools	› Rely on proof assistants to ensure the correctness of the proofs via computational checks.	› Pen-and-Paper

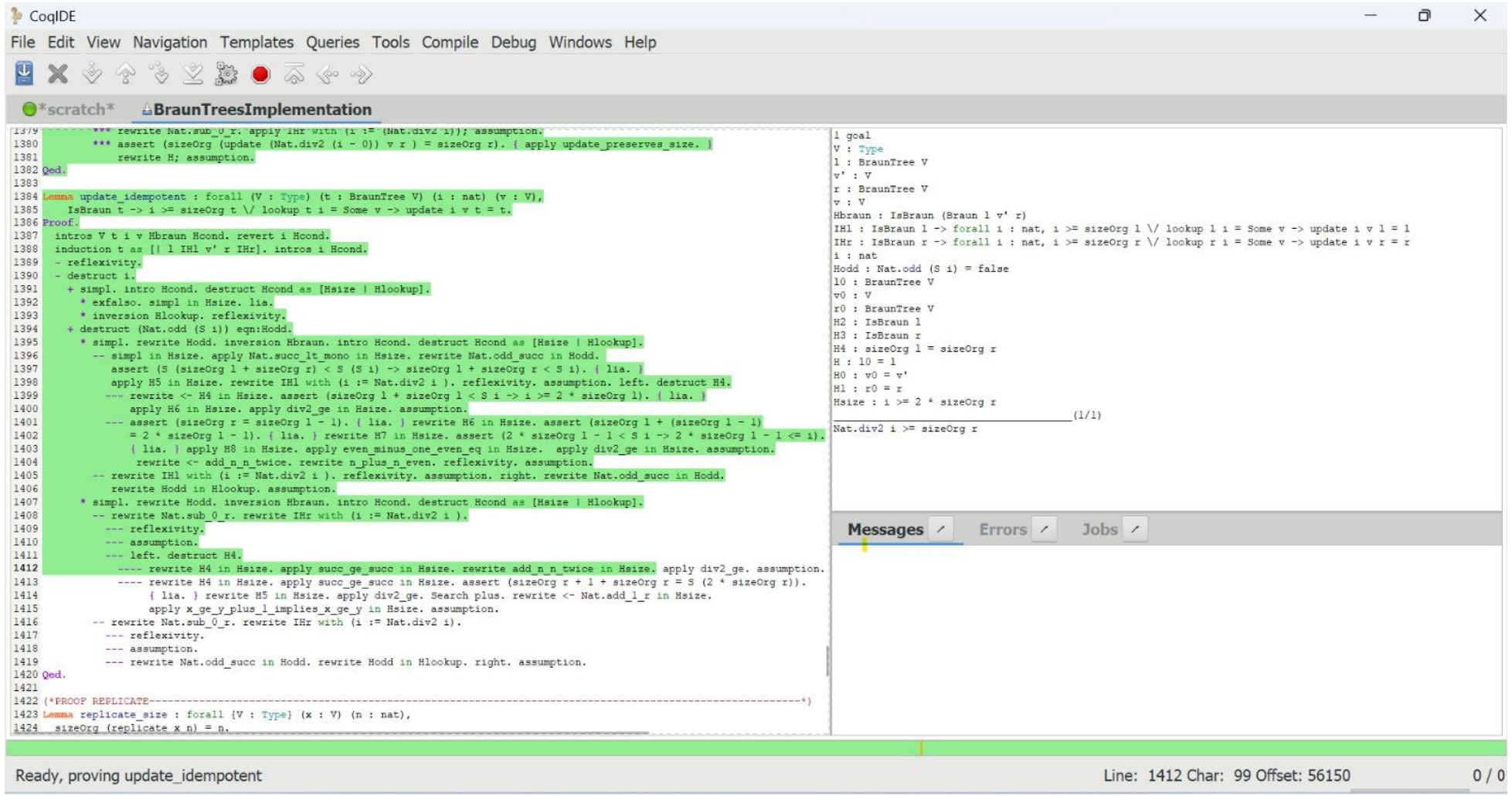
Coq and Gallina

Coq “is a formal proof management system. It provides a formal language to write mathematical definitions, executable algorithms and theorems”

Gallina is the said formal language that combines both a higher-order logic and a richly-typed functional programming language.

- defines functions or predicates and ensures logical correctness
- interactively develops **formal proofs**
- proofs developed using **Tactics**, eg.: apply lemmas, break down goals into subgoals, etc…
- extract programs to other languages: OCaml, Haskell

https://coq.inria.fr/about-coq


CoqIDE
File Edit View Navigation Templates Queries Tools Compile Debug Windows Help
scratch
BraunTreesImplementation
*** assert (sizeOrg (update (Nat.div2 (i - 0)) v r) = sizeOrg r). { apply update_preserves_size. }
rewrite H; assumption.
Qed.
Lemma update_idempotent : forall (V : Type) (t : BraunTree V) (i : nat) (v : V),
IsBraun t -> i >= sizeOrg t \/ lookup t i = Some v -> update i v t = t.
Proof.
intros V t i v Hbraun Hcond. revert i Hcond.
induction t as [| l IHl v' r IHr]. intros i Hcond.
- reflexivity.
- destruct i.
+ simpl. intro Hcond. destruct Hcond as [Hsize | Hlookup].
* exfalso. simpl in Hsize. lia.
* inversion Hlookup. reflexivity.
+ destruct (Nat.odd (S i)) eqn:Hodd.
* simpl. rewrite Hodd. inversion Hbraun. intro Hcond. destruct Hcond as [Hsize | Hlookup].
-- simpl in Hsize. apply Nat.succ_lt_mono in Hsize. rewrite Nat.odd_succ in Hodd.
assert (S (sizeOrg l + sizeOrg r) < S (S i) -> sizeOrg l + sizeOrg r < S i). { lia. }
apply H5 in Hsize. rewrite IHl with (i := Nat.div2 i). reflexivity. assumption. left. destruct H4.
--- rewrite <- H4 in Hsize. assert (sizeOrg l + sizeOrg l < S i -> i >= 2 * sizeOrg l). { lia. }
apply H6 in Hsize. apply div2_ge in Hsize. assumption.
--- assert (sizeOrg r = sizeOrg l - 1). { lia. } rewrite H6 in Hsize. assert (sizeOrg l + (sizeOrg l - 1)
= 2 * sizeOrg l - 1). { lia. } rewrite H7 in Hsize. assert (2 * sizeOrg l - 1 < S i -> 2 * sizeOrg l - 1 <= i).
{ lia. } apply H8 in Hsize. apply even_minus_one_even_eq in Hsize. apply div2_ge in Hsize. assumption.
rewrite <- add_n_n_twice. rewrite n_plus_n_even. reflexivity. assumption.
-- rewrite IHl with (i := Nat.div2 i). reflexivity. assumption. right. rewrite Nat.odd_succ in Hodd.
rewrite Hodd in Hlookup. assumption.
* simpl. rewrite Hodd. inversion Hbraun. intro Hcond. destruct Hcond as [Hsize | Hlookup].
-- rewrite Nat.sub_0_r. rewrite IHr with (i := Nat.div2 i).
--- reflexivity.
--- assumption.
--- left. destruct H4.
---- rewrite H4 in Hsize. apply succ_ge_succ in Hsize. rewrite add_n_n_twice in Hsize. apply div2_ge. assumption.
---- rewrite H4 in Hsize. apply succ_ge_succ in Hsize. assert (sizeOrg r + 1 + sizeOrg r = S (2 * sizeOrg r)).
{ lia. } rewrite H5 in Hsize. apply div2_ge. Search plus. rewrite <- Nat.add_1_r in Hsize.
apply x_ge_y_plus_1_implies_x_ge_y in Hsize. assumption.
-- rewrite Nat.sub_0_r. rewrite IHr with (i := Nat.div2 i).
--- reflexivity.
--- assumption.
--- rewrite Nat.odd_succ in Hodd. rewrite Hodd in Hlookup. right. assumption.
Qed.
(*PROOF REPLICATE
Lemma replicate_size : forall {V : Type} (x : V) (n : nat),
sizeOrg (replicate x n) = n.
1 goal
V : Type
l : BraunTree V
v' : V
r : BraunTree V
v : V
Hbraun : IsBraun (Braun l v' r)
IHl : IsBraun l -> forall i : nat, i >= sizeOrg l \/ lookup l i = Some v -> update i v l = l
IHr : IsBraun r -> forall i : nat, i >= sizeOrg r \/ lookup r i = Some v -> update i v r = r
i : nat
Hodd : Nat.odd (S i) = false
l0 : BraunTree V
v0 : V
r0 : BraunTree V
H2 : IsBraun l
H3 : IsBraun r
H4 : sizeOrg l = sizeOrg r
H : l0 = l
H0 : v0 = v'
H1 : r0 = r
Hsize : i >= 2 * sizeOrg r
(1/1)
Nat.div2 i >= sizeOrg r
Messages
Errors
Jobs
Ready, proving update_idempotent
Line: 1412 Char: 99 Offset: 56150
0 / 0

The CoqIDE

Proof Demo

Prove that the insert operation increases the size by 1:

```
Lemma size_insert_inc : forall (V: Type)
(v: V) (t: BraunTree V),
    sizeOrg (insert v t) = sizeOrg t + 1.
Proof.
```

```
1 goal
_____________________________________(1/1)
forall (V : Type) (v : V) (t : BraunTree V),
sizeOrg (insert v t) = sizeOrg t + 1
```

```
Lemma size_insert_inc : forall (V: Type)
(v: V) (t: BraunTree V),
    sizeOrg (insert v t) = sizeOrg t + 1.
Proof.
  intros V v t.
```

```
1 goal
V : Type
v : V
t : BraunTree V
_____________________________________(1/1)
sizeOrg (insert v t) = sizeOrg t + 1
```

Proof Demo

Definition of the Braun Trees:

```
Inductive BraunTree (V: Type) : Type :=
  | Empty
  | Braun (l: BraunTree V) (v: V) (r: BraunTree V).

  Arguments Empty {V}.
  Arguments Braun {V} _ _ _.
```

Inductive approach

```
Lemma size_insert_inc : forall (V: Type) (v: V)
(t: BraunTree V),
    sizeOrg (insert v t) = sizeOrg t + 1.
Proof.
  intros V v t. induction t as [|l IHl v' r IHr].
```

=

```
2 goals
V : Type
v : V
_____________________________________(1/2)
sizeOrg (insert v Empty) = sizeOrg Empty + 1
_____________________________________(2/2)
sizeOrg (insert v (Braun l v' r)) = sizeOrg (Braun l v' r) + 1
```

Proof Demo

Solve Empty Case:

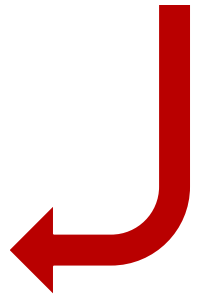
```
Lemma size_insert_inc : forall (V: Type) (v: V)
(t: BraunTree V),
    sizeOrg (insert v t) = sizeOrg t + 1.
Proof.
  intros V v t. induction t as [|l IHl v' r IHr].
  - simpl. reflexivity.
```

=

```
1 goal
V : Type
v : V
_____________________________________(1/1)
1 = 1
```

```
This subproof is complete, but there are some unfocused goals:

_____________________________________(1/1)
sizeOrg (insert v (Braun l v' r)) = sizeOrg (Braun l v' r) + 1
```



Proof Demo

Solve Recursive Case:

```
Lemma size_insert_inc : forall (V: Type) (v: V)
(t: BraunTree V),
    sizeOrg (insert v t) = sizeOrg t + 1.
Proof.
  intros V v t. induction t as [|l IHl v' r IHr].
  - simpl. reflexivity.
  - simpl. destruct (sizeOrg l =? sizeOrg r) eqn:Heq.
    + simpl.
```

```
1 goal
V : Type
v : V
l : BraunTree V
v' : V
r : BraunTree V
IHl : sizeOrg (insert v l) = sizeOrg l + 1
IHr : sizeOrg (insert v r) = sizeOrg r + 1
______________________________________(1/1)
sizeOrg
  (if sizeOrg l =? sizeOrg r
   then Braun (insert v l) v' r
   else Braun l v' (insert v r)) = S (sizeOrg l + sizeOrg r + 1)
```

Destruct tactic on size equality

```
1 goal
V : Type
v : V
l : BraunTree V
v' : V
r : BraunTree V
IHl : sizeOrg (insert v l) = sizeOrg l + 1
IHr : sizeOrg (insert v r) = sizeOrg r + 1
Heq : (sizeOrg l =? sizeOrg r) = true
______________________________________(1/1)
S (sizeOrg (insert v l) + sizeOrg r) = S (sizeOrg l + sizeOrg r + 1)
```

Proof Demo

Solve Recursive Case:

```
...
+ simpl.
  rewrite IHl.
  lia.
+ simpl.
  rewrite IHr.
  lia.
Qed.
```

Rewrite tactic

```
1 goal
V : Type
v : V
l : BraunTree V
v' : V
r : BraunTree V
IHl : sizeOrg (insert v l) = sizeOrg l + 1
IHr : sizeOrg (insert v r) = sizeOrg r + 1
Heq : (sizeOrg l =? sizeOrg r) = true
______________________________________(1/1)
S (sizeOrg l + 1 + sizeOrg r) = S (sizeOrg l + sizeOrg r + 1)
```

```
This subproof is complete, but there are some unfocused goals:

______________________________________(1/1)
sizeOrg (Braun l v' (insert v r)) = S (sizeOrg l + sizeOrg r + 1)
```

Inequality case

```
1 goal
V : Type
v : V
l : BraunTree V
v' : V
r : BraunTree V
IHl : sizeOrg (insert v l) = sizeOrg l + 1
IHr : sizeOrg (insert v r) = sizeOrg r + 1
Heq : (sizeOrg l =? sizeOrg r) = false
______________________________________(1/1)
S (sizeOrg l + sizeOrg (insert v r)) = S (sizeOrg l + sizeOrg r + 1)
```

```
1 goal
V : Type
v : V
l : BraunTree V
v' : V
r : BraunTree V
IHl : sizeOrg (insert v l) = sizeOrg l + 1
IHr : sizeOrg (insert v r) = sizeOrg r + 1
Heq : (sizeOrg l =? sizeOrg r) = false
______________________________________(1/1)
S (sizeOrg l + (sizeOrg r + 1)) = S (sizeOrg l + sizeOrg r + 1)
```

The Invariant

1. What Is an Invariant?
2. Implementation

```
Inductive IsBraun {V : Type} : BraunTree V -> Prop :=
  | IsBraun_Empty : IsBraun Empty
  | IsBraun_Node : forall l v r,
    IsBraun l ->
    IsBraun r ->
    (sizeOrg l = sizeOrg r \/ sizeOrg l = sizeOrg r + 1) ->
    IsBraun (Braun l v r).
```

3. Invariant’s role

Specification Approaches

Algebraic Specification

"equations that define how operations behave."

Model-Based Specification

"define a simpler or more abstract model and map our data structure to it."

```
forall (V : Type) (t : BraunTree V) (n : nat)
(v : V),
    sizeOrg (update n v t) = sizeOrg t.
```

```
Lemma size_list_equiv : forall (V : Type)
(t : BraunTree V),
    sizeOrg t = length (tree_to_list t).
```

List of Proofs

Maintaining invariant

```
› IsBraun t -> IsBraun (insert v t).
› IsBraun t -> removeRoot t = Some (v, t')
  -> IsBraun t'.
› IsBraun t -> forall t', remove t = Some t'
  -> IsBraun t'.
›  IsBraun t -> IsBraun (update i v t).
› IsBraun (replicate x n).
```

Algebraic

```
› sizeOrg (insert v t) = sizeOrg t + 1.
› sizeOrg (update n v t) = sizeOrg t.
› IsBraun t -> forall t', remove t = Some t'
  -> sizeOrg t' = sizeOrg t - 1.
› removeRoot (Braun l1 v1 r1) = Some (lv,
  newL) -> v1 = lv.
› IsBraun t -> remove t = Some newT -> lookup
  newT i = lookup t (S i).
› IsBraun t -> i >= sizeOrg t -> lookup t i =
  None.
› IsBraun t -> lookup t i = Some v -> i <
  sizeOrg t.
› IsBraun t -> lookup (insert v t) i = if i
  =? sizeOrg t then Some v else lookup t i.
› IsBraun t -> i >= sizeOrg t -> update i v t
  = t.
› IsBraun t -> lookup (update i v t) j = if
  (i =? j) && (i <? sizeOrg t) then Some v
  else lookup t j.
› IsBraun t -> i >= sizeOrg t \/ lookup t i =
  Some v -> update i v t = t...
```

Model-based

```
› sizeOrg t = length (tree_to_list t).
› IsBraun t -> tree_to_list (insert v t) =
  tree_to_list t ++ [v].
› IsBraun t -> forall t', removeRoot t =
  Some (v, t') -> tree_to_list t = v ::
  tree_to_list t'.
› IsBraun t -> lookup t i = nth_error
  (tree_to_list t) i.
› IsBraun t -> tree_to_list (update i v t)
  = upd (tree_to_list t) i v.
```

Learnings

1. Data Structures Wise
2. Formal Verification Wise
3. Practical Wise
4. Tool Wise
5. Tactics Wise
6. Research Wise

Any Questions?

Future Improvements

- Address the termination and decreasing argument issues
- Formulating the reverse of tree_to_list

Adding Efficiency:

- Complete the formal proof of Okasaki's algorithms by proving their equality with the non efficient algorithms:
  - Lemma okasaki_alg_1_proof : forall (V: Type) (t : BraunTree V),
    IsBraun t -> size t = sizeOkasaki t.
  - Lemma okasaki_alg_2_proof : forall (V: Type) (v: V) (n: nat),
    replicate v n = snd(replicateOkasaki v n).
  - Lemma okasaki_alg_3_proof : forall (V : Type) (l : list V),
    tree_to_list (listToBraun l) = tree_to_list (listToBraunOkasaki l).